NAME:-Shubham Suryakant
Biradar ROLL NO.:-A-12
BATCH:-A1
ASSINGMENT:-6

Aim:-Implementing Web Scraping in Python with BeautifulSoup.

THEORY:-

Q.1.What is web scraping? Explain the role of the Requests library in web scraping

Answer:-Web scraping is the process of automatically collecting data from websites using programs or scripts. It is used to extract information such as text, images, prices, tables, news, and other website content for analysis or storage. Web scraping saves time compared to manually copying data from webpages.

The Requests library in Python plays an important role in web scraping because it is used to send HTTP requests to websites and receive their webpage content. It helps fetch HTML data from a URL easily. With Requests, users can access webpages, download data, handle headers, cookies, and sessions. After getting the webpage content, other libraries like BeautifulSoup are commonly used to parse and extract useful information. Therefore, Requests is one of the basic and important libraries for web scraping in Python.

Q.2.What is BeautifulSoup? How does it help in parsing HTML?

Answer:-**BeautifulSoup** is a Python library used for **web scraping**. It helps extract data from HTML and XML documents easily. It creates a parse tree from webpage source code, making it simple to navigate and search elements.

How it helps in parsing HTML:

- It reads HTML content and organizes it into a structured format.
- It allows finding tags like <title>, <p>, , etc.
- It can extract text, links, images, and other data from webpages.
- It handles poorly formatted HTML better than many parsers.

Example: If a webpage has many paragraphs, BeautifulSoup can quickly find all <p> tags and extract their text.

Q.3.Differentiate between web scraping and web crawling. **Answer:-****Difference between Web Scraping and Web Crawling:**

Basis	Web Scraping	Web Crawling
Meaning	Extracting specific data from websites.	Browsing and indexing multiple web pages automatically.
Purpose	To collect useful information like prices, emails, text, etc.	To discover pages and gather links for search engines.
Focus	Data extraction from selected pages.	Navigation through many pages on the internet.
Example	Collecting product prices from shopping sites.	Google bot scanning websites for search results.
Tools Used	BeautifulSoup, Scrapy, Selenium	Search engine bots, Scrapy crawlers

In short:

- **Web Scraping** = Getting data from webpages.
- **Web Crawling** = Exploring webpages and finding links/data.

**Q.4.What are the commonly used methods in BeautifulSoup? **

Answer:-**Commonly Used Methods in BeautifulSoup:**

1. **find()** – Finds the first matching tag.
2. **find_all()** – Finds all matching tags.
3. **select()** – Finds elements using CSS selectors.

4. **select_one()** – Finds the first element using CSS selector.
5. **get_text()** – Extracts text from tags.
6. **get()** – Gets the value of an attribute like `href` or `src`.
7. **prettify()** – Formats HTML in a readable way.
8. **title** – Gets the title tag of the page.
9. **parent** – Accesses the parent tag.
10. **children** – Accesses child tags.

Example:

- `find('p')` → Finds first paragraph tag.
- `find_all('a')` → Finds all links.
- `get_text()` → Extracts only text.

Q.5.What are the ethical and legal considerations in web scraping?

Answer:-Ethical and Legal Considerations in Web Scraping:

1. **Respect Website Terms of Service** – Many websites have rules about scraping. Always check and follow their terms.
2. **Check robots.txt File** – Websites may specify which pages can or cannot be scraped. Respect these rules.
3. **Do Not Overload Servers** – Sending too many requests quickly can slow down or crash a website. Use reasonable request rates.
4. **Respect Privacy** – Do not collect personal, private, or sensitive user data without permission.
5. **Copyright Issues** – Website content such as text, images, and data may be protected by copyright. Do not copy or reuse without permission.
6. **Use Data Responsibly** – Collected data should not be used for spam, fraud, or harmful purposes.
7. **Identify Yourself if Needed** – Use a clear user-agent and be transparent when appropriate.
8. **Follow Laws and Regulations** – Different countries have laws related to data protection and unauthorized access.

```
#CODE
import requests
from bs4 import BeautifulSoup
# Step 1: Define the URL
url = "https://example.com"
# Step 2: Send HTTP request
response = requests.get(url)
# Step 3: Check if request was successful
if response.status_code == 200:
    print("Successfully fetched the webpage!\n")
    # Step 4: Parse HTML content
    soup = BeautifulSoup(response.text, "html.parser")
    # Step 5: Extract all links (anchor tags)
    links = soup.find_all('a')
    print("Links found on the webpage:\n")
    for link in links:
        text  =  link.get_text()
        href = link.get('href')
        print("Text:",    text)
        print("Link:",    href)
        print("-" * 40)
else:
    print("Failed to retrieve webpage. Status code:", response.status_code)
```

```
Successfully fetched the webpage!
```

```
Links found on the webpage:

Text: Learn more
Link: https://iana.org/domains/example
----------------------------------------
```

CONCLUSION:- The aim of implementing web scraping in Python with **BeautifulSoup** was successfully achieved. BeautifulSoup is a useful library for extracting data from HTML and XML web pages in a simple and efficient way. It helps in finding tags, elements, text, links, and other webpage content easily. By using Python along with BeautifulSoup, we can automate the process of collecting data from websites instead of doing it manually. This saves time, reduces effort, and improves accuracy. Overall, BeautifulSoup is an effective tool for web scraping and data collection tasks in Python.